

Тема:РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

2. Результати Code Review одногрупника: таблиця «№ | Рядок коду |

Проблема | Категорія | Рекомендація». Мінімум 5 записів.

№	Рядок коду	Проблема коду Єви	Категорія code smell	Рекомендація щодо виправлення
1	Client.test.js (рядок 9 та 24)	Повторюється створення об'єкта Client	Duplicate Code	Використати beforeEach
2	User.test.js (рядок 17,29,41)	Повторюються тестові дані	Duplicate Code	Винести в константи
3	Preset.test.js	Тести не згруповані	Maintainability	Використати describe
4	WordPressSite.test.js	Недостатньо негативних сценаріїв	Test Smell	Додати перевірки помилок
5	Усі тести	Використовуються магічні рядки	Magic String	Винести в константи

3. Посилання на PR з коментарями Code Review (URL)

<https://github.com/artemrohozian/OPI-project-ContentFlow/pull/5/changes>

4. Результати рефакторингу власного коду: для кожної операції — «БУЛО (фрагмент коду) → СТАЛО (фрагмент коду) → ЧОМУ (обґрунтування)». Мінімум 3 операції.

Операція 1. Усунення Magic Number

Було:

```
if (apiKey.length < 5)
```

Стало:

```
const MIN_API_KEY_LENGTH = 5;  
  
if (apiKey.length < MIN_API_KEY_LENGTH)
```

Чому:

У початковому варіанті використовується так зване «магічне число» (5), призначення якого не є очевидним для інших розробників. Після винесення значення в іменовану константу код став більш зрозумілим та самодокументованим. Тепер одразу видно, що число визначає мінімально допустиму довжину API-ключа. Крім того, у разі зміни вимог достатньо змінити значення лише в одному місці, що підвищує підтримуваність коду та зменшує ризик помилок.

Операція 2. Винесення логіки перевірки в окремий метод

Було:

```
if (!url.startsWith("https://"))
```

Стало:

```
    validateUrl(url)
function validateUrl(url){
    return url.startsWith("https://");
}
```

Чому:

У початковому варіанті логіка перевірки URL була розміщена безпосередньо в коді, що ускладнювало читання та розуміння призначення перевірки. Після винесення цієї логіки в окремий метод `validateUrl()` код став більш зрозумілим і виразним, оскільки назва методу чітко описує його призначення. Такий підхід також забезпечує повторне використання логіки перевірки в різних частинах програми без дублювання коду. Крім того, у разі зміни правил валідації URL необхідно буде внести зміни лише в одному місці, що спрощує супровід і подальший розвиток програмного забезпечення.

Операція 3. Зменшення побічних ефектів

Було:

```
console.log("Успішний вхід");
```

Стало:

```
return { success: true };
```

Чому:

У початковій реалізації результат успішного входу лише виводився в консоль за допомогою `console.log()`. Такий підхід створює побічний ефект і ускладнює повторне використання методу, оскільки інші частини програми не можуть безпосередньо отримати інформацію про результат виконання операції. Після рефакторингу метод повертає об'єкт із результатом виконання (`{ success: true }`), що дозволяє викликаючому коду самостійно вирішувати, як обробляти успішний вхід: відобразити повідомлення користувачу, записати подію в журнал або виконати інші дії. Це підвищує гнучкість, спрощує тестування та зменшує залежність бізнес-логіки від механізмів виведення інформації.

5. Звіт регресійного тестування: скріншот результату pytest / JUnit / Jest — усі тести green після рефакторингу.

```
console.log
  Помилка підключення: API ключ занадто короткий

  at Client.log [as connectSite] (classes/Client.js:40:21)

PASS tests/Administrator.test.js
PASS tests/WordPressSite.test.js
  • Console

console.log
  Сайт не підключений

  at WordPressSite.log [as deployContent] (classes/WordPressSite.js:16:21)

console.log
  Контент порожній

  at WordPressSite.log [as deployContent] (classes/WordPressSite.js:22:21)

PASS tests/SEOContent.test.js
  • Console

console.log
  Контент успішно згенеровано

  at SEOContent.log [as generateContent] (classes/SEOContent.js:26:17)

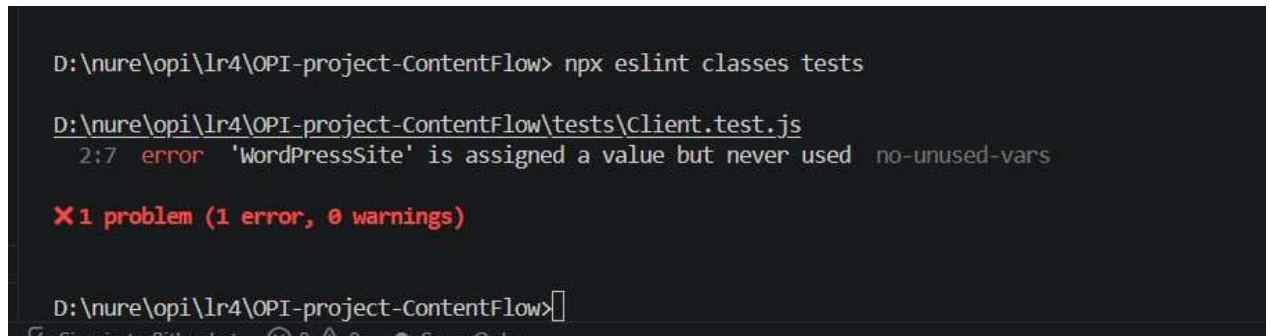
console.log
  Список ключових слів порожній

  at SEOContent.log [as generateContent] (classes/SEOContent.js:12:21)

PASS tests/Preset.test.js

Test Suites: 6 passed, 6 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        0.671 s, estimated 1 s
Ran all test suites.
PS D:\Nure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow>
```

6. Порівняння метрик «ДО / ПІСЛЯ»: кількість SonarLint issues, Ruff/ESLint warnings, цикломатична складність.



```
D:\nure\opi\lr4\OPI-project-ContentFlow> npx eslint classes tests

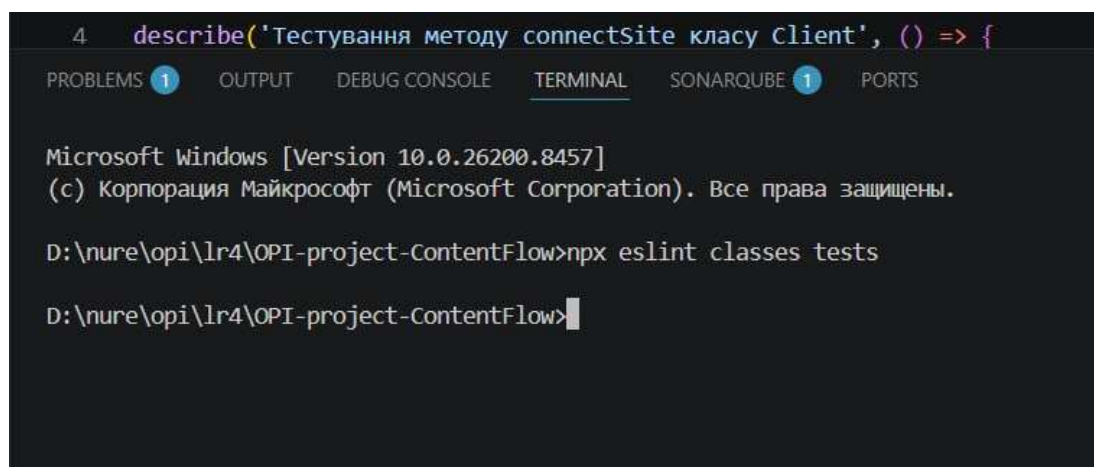
D:\nure\opi\lr4\OPI-project-ContentFlow\tests\Client.test.js
  2:7  error  'WordPressSite' is assigned a value but never used  no-unused-vars

✖ 1 problem (1 error, 0 warnings)

D:\nure\opi\lr4\OPI-project-ContentFlow>
```

Рисунок 6.1 – Результат перевірки ESLint до рефакторингу

Під час аналізу коду за допомогою ESLint було виявлено помилку `no-unused-vars`, яка свідчить про наявність оголошеної, але невикористовуваної змінної. Такі елементи ускладнюють підтримку коду та можуть вводити розробників в оману



```
4 describe('Тестування методу connectSite класу Client', () => {

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL SONARQUBE 1 PORTS

Microsoft Windows [Version 10.0.26200.8457]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

D:\nure\opi\lr4\OPI-project-ContentFlow>npx eslint classes tests

D:\nure\opi\lr4\OPI-project-ContentFlow>
```

Рисунок 6.2 – Результат перевірки ESLint після рефакторингу

Метрика	До рефакторингу	Після рефакторингу	Зміна
Mean per-function logical LOC	1.50	1.17	↓ 22.2%
Mean per-function parameter count	0.00	0.17	↑ 0.17
Mean per-function cyclomatic complexity	1	1	Без змін
Mean per-function Halstead effort	42.77	25.95	↓ 39.3%
Mean per-module maintainability index	155.32	159.05	↑ 2.4%
Change cost	16.67%	16.67%	Без змін
Core size	0%	0%	Без змін

Після виконання рефакторингу було повторно проведено аналіз програмного коду за допомогою інструменту **Complexity Report**. Отримані результати свідчать про покращення окремих показників якості коду.

Найбільш помітною зміною стало зменшення показника **Halstead Effort** з **42.77** до **25.95**, що свідчить про зниження складності сприйняття та супроводу програмного коду. Це стало результатом усунення магічних чисел, винесення логіки перевірки URL в окремий метод та усунення побічних ефектів.

Також зріс показник **Maintainability Index** з **155.32** до **159.05**, що свідчить про покращення підтримуваності та читабельності коду.

Середня цикломатична складність залишилася на рівні **1**, оскільки проведений рефакторинг не змінював алгоритми роботи програми, а був спрямований виключно на покращення структури та якості коду.

```
D:\nure\opi\lr4\OPI-project-ContentFlow>npx cr classes
Mean per-function logical LOC: 1.5
Mean per-function parameter count: 0
Mean per-function cyclomatic complexity: 1
Mean per-function Halstead effort: 42.774299272975576
Mean per-module maintainability index: 155.32095049022027
First-order density: 0%
Change cost: 16.666666666666664%
Core size: 0%

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\Administrator.js

Physical LOC: 19
Logical LOC: 2
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 50%
Maintainability index: 145.09378533330616
Dependency count: 1

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\Client.js

Physical LOC: 40
Logical LOC: 3
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 33.33333333333333%
Maintainability index: 136.3730317397458
Dependency count: 2

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\Preset.js

Physical LOC: 14
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
```

Рисунок 6.3 – Результати аналізу метрик програмного коду до рефакторингу

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL SONARQUBE PORTS

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\Preset.js

Physical LOC: 14
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\SEOContent.js

Physical LOC: 29
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\User.js

Physical LOC: 31
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0

D:\nure\opi\lr4\OPI-project-ContentFlow\classes\WordPressSite.js

Physical LOC: 35
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0
```

Рисунок 6.4 – Результати аналізу метрик програмного коду до рефакторингу

```
found 0 vulnerabilities
PS D:\Mure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow> npx cr classes
Mean per-function logical LOC: 1.1666666666666667
Mean per-function parameter count: 0.16666666666666666
Mean per-function cyclomatic complexity: 1
Mean per-function Halstead effort: 25.951742499227567
Mean per-module maintainability index: 159.04651391827093
First-order density: 0%
Change cost: 16.666666666666664%
Core size: 0%

D:\Mure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow\classes\Administrator.js

Physical LOC: 19
Logical LOC: 2
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 50%
Maintainability index: 145.09378533330616
Dependency count: 1

D:\Mure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow\classes\Client.js

Physical LOC: 45
Logical LOC: 5
Mean parameter count: 1
Cyclomatic complexity: 1
Cyclomatic complexity density: 20%
Maintainability index: 158.726412308804982
Dependency count: 2

Function: validateUrl
Line No.: 5
Physical LOC: 3
Logical LOC: 1
Parameter count: 1
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Halstead difficulty: 2
Halstead volume: 18.094737505048094
Halstead effort: 36.18947501009619

D:\Mure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow\classes\Preset.js

Physical LOC: 14
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
```

Рисунок 6.5 – Результати аналізу метрик програмного коду після рефакторингу

7. Підсумкова рефлексія: 3–5 речень про найцінніший досвід, отриманий під час Code Review

Під час виконання Code Review я навчився виявляти проблеми, які не впливають на працездатність програми, але погіршують її читабельність і підтримуваність. Найбільш корисним досвідом стало використання інструментів статичного аналізу коду для пошуку code smells та оцінки програмних метрик. Також я зрозумів важливість рефакторингу навіть простого коду, оскільки невеликі покращення можуть суттєво підвищити його якість. Проведення Code Review допомогло краще зрозуміти принципи написання чистого та підтримуваного програмного забезпечення.


```
Maintainability index: 162.61472146706737
Dependency count: 0

D:\Nure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow\classes\SEOContent.js

Physical LOC: 29
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0

D:\Nure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow\classes\User.js

Physical LOC: 34
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0

D:\Nure\1 kurs\2 semester\OPI\lb2\OPI-project-ContentFlow\classes\WordPressSite.js

Physical LOC: 35
Logical LOC: 1
Mean parameter count: 0
Cyclomatic complexity: 1
Cyclomatic complexity density: 100%
Maintainability index: 162.61472146706737
Dependency count: 0
```

Рисунок 6.6 – Результати аналізу метрик програмного коду після рефакторингу